

craftService

```
const { injectCounter } = craftService(  
  { name: 'Counter', scope: 'function' },  
  ({ initialValue }: { initialValue: number }) =>  
    state(initialValue, ({ update }) => ({  
      increment: () => update((v) => v + 1),  
    })),  
);
```



**@craft-ng - Type-Safe Reactive States
Management for Angular**
Promotes declarative and composable
code with logic management utilities
Par Romain Geffrault



craftService

Des services Angular composables et testables de bout en bout

@craft-ng repose sur l'injection de dépendances d'Angular.

Pas de magie noire. Pas de runtime custom.

Juste une couche déclarative par-dessus le DI que vous connaissez.



1. Un service simple avec craftService

Un service = un nom, un scope, une factory

```
import { craftService, state } from '@craft-ng/core';

const { injectCounter, provideCounter } = craftService(
  { name: 'Counter', scope: 'toProvide' },
  () =>
    state(0, ({ update }) => ({
      increment: () => update((v) => v + 1),
      decrement: () => update((v) => v - 1),
    })),
);
```

`injectCounter()` injecte le service dans un composant.
`provideCounter()` le fournit dans le module/route.

Le scope `toProvide` = fournir explicitement.

Le scope `global` = singleton (`providedIn: 'root'`).

Derrière, c'est du pur Angular DI.



2. Composer des services : yield*

La composition se fait via une **fonction génératrice**.

Pas besoin d'en savoir plus sur les générateurs.

Retenez juste ce pattern :

```
// Angular classique
const api = inject(ApiService);

// Avec @craft-ng
const api = yield * ApiServiceToYield();
```

C'est la même chose.

yield* injecte un craftService dans un autre craftService.



3. yield* en contexte

```
const { CounterToYield } = craftService(
  { name: 'Counter', scope: 'toProvide' },
  () =>
    state(0, ({ update }) => ({
      increment: () => update((v) => v + 1),
    })),
);

const { injectCounterFacade, provideCounterFacade } = craftService(
  { name: 'CounterFacade', scope: 'toProvide' },
  function* () {
    const counter = yield* CounterToYield();

    return {
      read: () => counter(),
      increment: () => counter.increment(),
    };
  },
);
```

function* signale qu'il y a des dépendances.

Chaque yield* est tracké par le système de types.

Résultat : un arbre de dépendances complet, inféré automatiquement.



4. Le système d'inputs

Un craftService peut déclarer des **inputs typés** dans sa factory :

```
const { injectCounter, CounterToYield } = craftService(  
  { name: 'Counter', scope: 'function' },  
  (inputs: { initialValue: MaybeSignal<number> }) =>  
    state(toValue(inputs.initialValue), ({ update }) => ({  
      increment: () => update((v) => v + 1),  
    })),  
  ).
```

À l'injection, les inputs sont passés directement :

```
// Dans un composant  
const counter = injectCounter({ initialValue: 10 });  
  
// Ou avec un signal pour du binding réactif  
const initial = signal(0);  
const counter = injectCounter({ initialValue: initial });
```

Et lors de la composition via `yield*` :

```
const { injectExtended } = craftService(  
  { name: 'Extended', scope: 'global' },  
  function* () {  
    const c1 = yield* CounterToYield({ initialValue: signal(10) })  
    const c2 = yield* CounterToYield({ initialValue: signal(20) })  
  
    return { c1, c2 }; // 2 instances distinctes  
  },  
).
```

Les inputs sont obligatoires : oublier un input = erreur à l'exécution.
Le type force la complétion.



5. Exposition partielle

Vous pouvez choisir exactement ce que vous exposez d'une dépendance :

```
const { injectCounterExtended } = craftService(  
  { name: 'CounterExtended', scope: 'toProvide' },  
  function* () {  
    return yield* CounterToYield(undefined, ({ $self, increment  
      $self,  
      incrementCounter: increment,  
    }));  
  },  
);
```

`$self` = la valeur callable du signal (`counter()`).

`increment` = renommé en `incrementCounter`.

`decrement` = invisible, non exposé.

Le type résultant ne contient que ce qui est déclaré.

Un double intérêt pour les tests : on peut mocker uniquement ce qui est exposé, et le typage nous protège des changements d'implémentation.



6. Tester avec `setupCraftServiceTestingByRegister`

Et c'est là que ça devient intéressant.

L'arbre de dépendances inféré par `yield*` sert directement à construire un registre de test exhaustif.

```
import { setupCraftServiceTestingByRegister } from '@craft-ng/core';

const { sut, mocks } = setupCraftServiceTestingByRegister(injector, {
  CounterFacade: provideCounterFacade(),
  Counter: {
    $self: vi.fn(() => 42),
    increment: vi.fn(),
  },
});

expect(sut.read()).toBe(42);
sut.increment();
expect(mocks.Counter.increment).toHaveBeenCalledTimes(1);
```

`sut` = le service sous test, injecté dans un vrai TestBed Angular.

`mocks` = uniquement les dépendances mockées dans le registre.



7. Couverture obligatoire (type-safe)

Le système de types **force** un registre plat et exhaustif pour tout l'arbre atteignable.

```
// ✗ Erreur de compilation  
setupCraftServiceTestingByRegister(injectCounterFacade, {  
    CounterFacade: provideCounterFacade(),  
    // Counter manque  
});
```

Chaque nœud doit être déclaré explicitement :

- soit par un mock brut { ... } (typé)
- soit par 'real'
- soit par le vrai provider provideCounter()
- soit par 'notReached' si une branche est coupée par un ancêtre mocké

Plus de zones implicites : le registre décrit tout l'état du graphe de test.



8. Mock, real ou vrai provider ?

Le registre vous oblige à préciser le statut de chaque service.

```
// Mock brut = la branche entière est coupée
setupCraftServiceTestingByRegister(injectRootCounter, {
  RootCounter: provideRootCounter(),
  ParentCounter: { increment: vi.fn() },
  ChildCounter: 'notReached',
});
```

```
// 'real' = on garde l'implémentation réelle d'un global/fonction
setupCraftServiceTestingByRegister(injectCounterFacade, {
  CounterFacade: provideCounterFacade(),
  Counter: 'real',
});
```

```
// Vrai provider = les enfants restent atteignables
setupCraftServiceTestingByRegister(injectRootCounter, {
  RootCounter: provideRootCounter(),
  ParentCounter: provideParentCounter(),
  ChildCounter: {
    $self: vi.fn(() => 0),
    increment: vi.fn(),
  },
});
```

Le typage s'adapte dynamiquement à votre stratégie de test, mais rien n'est caché.



9. Les scopes disponibles

Chaque `craftService` déclare un **scope** qui définit son cycle de vie :

- **global** — Singleton, fourni automatiquement à la racine (= `providedIn: 'root'`). Aucun provider explicite nécessaire.
- **toProvide** — Doit être fourni explicitement via `provideX()`. Idéal pour les services liés à une route ou un composant.
- **manuallyProvidedAtRoot** — Singleton à la racine, mais fourni manuellement. Permet à ces service d'être injectés dans des services `global`. Mais restent obligatoirement fournis dans les tests.
- **function** — Nouvelle instance à chaque injection. Aucun singleton, aucun partage.
- **abstract** — Déclare un contrat sans implémentation. Expose `XRequirement` pour obliger une implémentation concrète plus tard.

Chaque scope impacte directement le comportement de `setupCraftServiceTestingByRegister` (provider réel, `real`, mock ou exigence de couverture).

On détaillera chaque scope avec des exemples concrets dans un prochain post.



10. Résumé

`craftService + setupCraftServiceTestingByRegister :`

- ✓ Basé sur le DI Angular natif
- ✓ Inputs natifs des fonctions et typées
- ✓ Composition via `yield*` (= `inject()`)
- ✓ Arbre de dépendances inféré par les types
- ✓ Exposition partielle typée
- ✓ Registre de test exhaustif et typé

📖 Doc : <https://ng-angular-stack.github.io/craft/>



**Merci pour la lecture !
Dis-moi ce que t'en
penses en commentaire.**



Romain Geffrault
Développeur Angular
Suis moi pour du contenu dédié à
Angular